

supports Next.js/Vite/Angular, secure sandbox via Service Workers, instant dependency resolution, collaboration via share links, limited for binaries needing native syscalls.

Glitch: Containerised node (and limited Python) apps auto-restart on edit, collaborative CRDT editor, instant public domain, simple `.env` handling, scheduled wake/always-on paid boost, asset CDN, version remix/fork model, basic AI suggestions, logging console, constrained memory/CPU, not ideal for multi-service production topologies.

CodePen: Browser tri-pane editor (HTML/CSS/JS) with preprocessors (SCSS, Babel, TypeScript), asset hosting (Pro limits higher), instant rerender, CSS/JS settings per pen, embeddable iframes, collection and fork workflow, limited collaboration (comments, professor mode), no backend runtime, export zip for deployment.

Google Colab: Managed Jupyter-like notebooks with ephemeral Debian VM, Python kernels, GPU/TPU acceleration (quota managed), file backing via Drive, pip/apt installs, AI code completions, notebook sharing + comments, integration with BigQuery/Vertex SDKs, session timeouts (idle, 12h), limited secrets handling, suited for data/ML experiments.

AWS Cloud9: Cloud-hosted IDE on EC2, sudo-level Linux shell, IAM-integrated credentials, AWS CLI/CDK/SAM preconfigured, CodeWhisperer AI suggestions, pair edit cursors, environment hibernation to save cost, port preview with HTTPS, integrated debugger/terminal, direct Lambda packaging, latency depends on region, vendor lock-in risk.

Zed: Native Rust-based editor leveraging incremental GPU rendering for ultra-low latency, tree-sitter parsing, multi-buffer semantic indexing, AI completion/chat, real-time presence, voice rooms, CRDT sync, runs local toolchains, integrates Git, lacks built-in cloud execution/hosting features.

Table 1 illustrates the top differences between these platforms.

Testing and validation after vibe coding

After a fast ‘vibe coding’ spike (e.g., hacking a real time chat prototype in Replit or a React dashboard in StackBlitz), formal testing/validation means first freezing the environment (pin package versions, create a devcontainer), and then layering quality gates: run linters/types (ESLint, mypy), secret/licence and vulnerability scans (Trivy, npm audit), write/curate unit tests around emergent utilities (e.g., message serializer), add contract tests against external APIs (mock Slack webhook), and a slim end-to-end flow (user sends message → appears in room). Next, regenerate or refine any AI-authored code/tests and review for logic gaps. Capture golden snapshots (playwright visual baseline, JSON fixture for `/messages` endpoint) to detect regressions. Introduce performance and accessibility smoke tests (Lighthouse budget for the React page, p95 latency assertion for message publish). Stand up ephemeral preview environments per pull request to validate behaviour in near-production settings. Instrument telemetry (OpenTelemetry traces, custom error counter) and define an error budget. Run basic dynamic security probes (OWASP ZAP against exposed endpoints). Only after all gates pass do you merge and promote, preserving traceable artifacts (test reports, SBOM) so the once fluid ‘vibe’ code becomes a reproducible, auditable, production-ready service.

How to adopt vibe coding in an organisation

Define strategic purpose: Articulate why rapid, flow-centric development matters (e.g., accelerate hypothesis validation, reduce concept-to-PoC latency). Tie to measurable OKRs (cycle time, validated experiments per quarter).

Classify work streams: Segregate ‘exploratory spike’, ‘accelerated prototype’, and ‘production hardening’ phases with explicit exit criteria (test coverage thresholds, security sign-offs, performance budgets).

Establish tooling portfolio:

Approve a curated stack (e.g., StackBlitz/Replit for spikes, Codespaces/devcontainers for convergence, mainline IDE + CI for hardening). Document sanctioned usage patterns, data boundaries, and supported languages.

Standardise environment

transition: Mandate a conversion checklist: lock dependencies, generate devcontainer, add baseline lint/type/security config, provision ephemeral preview infrastructure.

Codify AI assistance policy:

Define permitted AI models, logging boundaries, PII handling, and required human review depth for AI-generated code/tests. Enforce traceability tags in commit messages (e.g., ‘AI-CoAuthored’).

Implement lightweight governance

gates: Introduce progressive gates:

- *Gate 1 (Spike):* Security lint + licence scan only.
- *Gate 2 (Prototype):* Unit tests + contract tests + SBOM.
- *Gate 3 (Pre-prod):* Full CI matrix, SAST/DAST, performance smoke, accessibility.

Embed security and compliance

early: Integrate secret detection, dependency risk scoring, and automated SBOM generation inside the fast-feedback loop; block escalation if unresolved critical issues persist beyond the defined SLA.

Enforce observability from

prototype onward: Require minimal telemetry (traces, key business metrics, error rate) even in spikes to prevent refactoring churn and enable early performance baselining.

Institutionalise rapid knowledge

capture: After each spike, store a concise design delta record